

What is System Designing ?

- System Design is the blueprint for how a system works. It defines the Architecture (Structure), Modules (Parts), and Data Flow (Communication).

Types of System Design

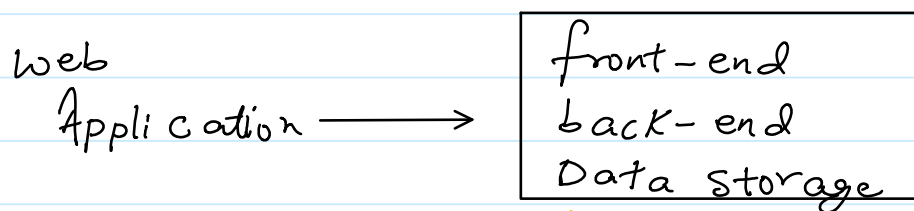
- i) LLD { Low level Design }
- ii) HLD { High level Design }

HLD	LLD
☆ Describe the main components that would be developed for the resulting product.	Describe the design of each element mentioned in the High-Level Design of the system
☆ The system architecture details, database design, services and processes, the relationship between various modules, and features.	Classes, interfaces, relationships between different classes, and actual logic of the various components.

Architecture $\Rightarrow$ Internal design details for building the applications.

Monolithic Architecture

- If all the components and functionalities of a project are entangled and combined in a single codebase, then that is a monolithic application.
- Monolithic Architecture has less complexity $\Rightarrow$ Easier to understand $\Rightarrow$ Higher productivity.



$\hookrightarrow$ Written & Deployed together

- Monolithic system is also known as centralized system.

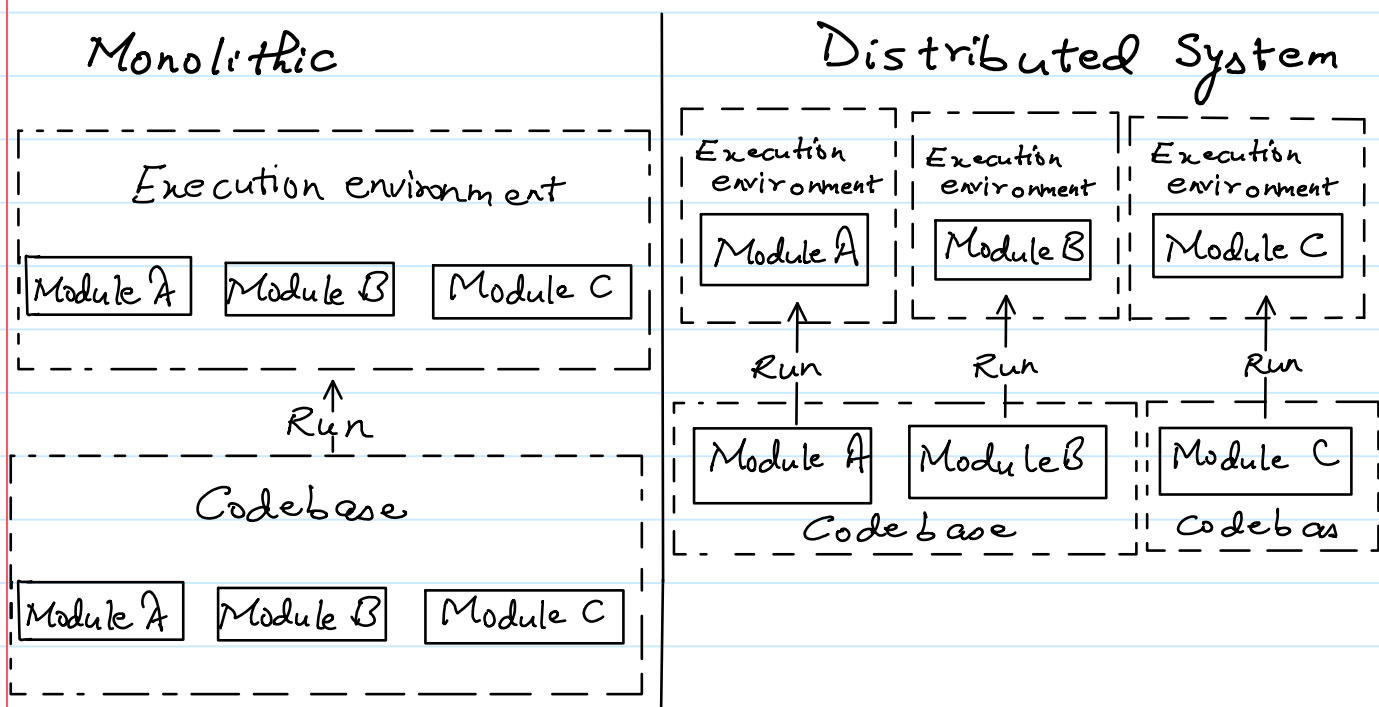
Advantages/when it is good

1. When we are just starting.
2. In monolithic architecture, all the modules are present in the single system, so they require fewer network calls as compared to other architectures.
3. It is comparatively easier to secure monolithic system.
4. Integration testing is easier.
5. Less confusion.

Disadvantages

1. In monolithic architecture, every module is combined in a single system, so if there is an error or bug in a single module, it can destroy the complete system.

2. In monolithic architecture, whenever a single module is updated, the whole system needs to be updated to reflect the changes to the users. All modules are present in a single system and are connected to one another, so the whole system needs to be updated.
3. In monolithic architecture, if there is any change in a single module's programming language or framework, it will affect the entire system. The entire system needs to be changed because every module is interlinked and tightly coupled.



- **Distributed system:** A distributed system is a collection of multiple individual systems connected through a network that share resources, communicate and coordinate to achieve common goals.

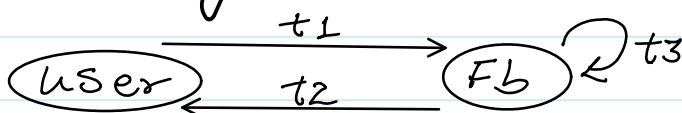
Advantages

1. Scalable $\rightarrow$ Horizontal scale
2. No single point of failure. for eg $\rightarrow$ if user service is not working it will not affect authentication service, product service, order service etc.
3. Low latency

Disadvantages

1. Complex, because many services communicate among them.
2. Management required like load balancing
3. Difficult to secure
4. Message may be lost in between nodes.

Latency



Latency = Network delay + Computational delay

$$\Delta t = t_1 + t_2 + t_3$$

Monolithic

$$\Delta t = \text{Computational Delay}$$

Distributed

$$\Delta t = \text{Comp. delay} + \text{Network delay}$$

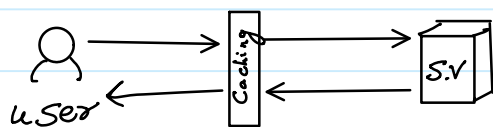
Reducing latency

23 December 2025

08:32 PM

Reducing latency

- 1) Caching 2) CDN 3) Upgrading system



Caching vs CDN

- CDNs are geographically distributed networks of proxy servers and their objective is to serve content to users more quickly.
- Caching is the process of storing information for a set period of time on a computer.

Throughput

The volume of work or information flowing through a system. Throughput is the amount of data transmitted per unit of time. It is the process flow rate. Throughput is measured in bits per second i.e bps

- Distributed system will have more throughput than monolithic application because:
 - i> No limit on resources
 - ii> Load balancing

Causes of Low throughput

- i> latency ii> protocol overhead iii> Congestion

Improving throughput

- i) CDN ii) Caching iii) Distributed system
- iv) load balancing v) upgrading system

Availability, Replication & Redundancy

1. What is Availability?

- Definition: Availability is the percentage of time a system is operational and accessible to perform its intended function.
- Concept: It essentially measures "uptime". A highly available system is one that is accessible almost all the time, even during components failure
- Formula:

$$\text{Availability} = \frac{\text{Uptime}}{\text{Total time}} \times 100\%$$

$$\{\text{total time} = \text{uptime} + \text{downtime}\}$$

2. What is Redundancy?

- Definition: Redundancy is the practice of duplicating critical components or functions of a system so if one fails, another can take place
- Goal: To remove Single point of failure (SPOF)
- Types of redundancy:
 - i) Hardware Redundancy: {Extra Hard drive, servers}
 - ii) Software Redundancy: {Running multiple instances}

Replication

23 December 2025 09:13 PM

Q. What is Replication ?

- Definition: Replication refers to creating and maintaining exact copies of data across multiple servers or databases.

- Goal:

- Fault Tolerance: If one database node goes down, data is still safe and accessible from another.

- Performance: Read requests can be distributed (load balanced) across multiple replicas to reduce latency.

- Disaster Recovery: Keep data in different geographical regions.

Fault Tolerance $\propto$ Availability

How to increase Availability

- i) Replication
- ii) Distributed system
- iii) Redundancy

Consistency:

Q. What is consistency ?

Definition: Consistency ensures that every read request returns the most recent write. Regardless of which node or server a client connects to, they should see the exact same data.

Consistency

26 December 2025 08:23 PM

Dirty Read: When a client reads data that is outdated or has not yet been fully committed across all systems, it is called "Dirty Read".

Real-world example:

- Bank Balance: If you deposit ₹100, the ATM should immediately show the updated balance.
- Movie Ticket: If user A books the last seat on Delhi server, User B on Pune server should immediately see it as "Booked" to prevent double booking.

2. Monolithic vs Distributed Systems

- Monolithic: Generally natively consistent because there is often only one database instance. All requests go to the same source, so no synchronization is needed.
- Distributed: Inherently prone to inconsistency because data is replicated across different servers.
 - sync time (t): If server A updates at t_1 and server B takes time to sync t_2 , the system is inconsistent for the duration $\max(t_1, t_2)$.

3. Factors Improving Consistency

- a) Increase network speed: Faster synchronization between nodes reduces the window of inconsistency.
- b) Stop Read operations: Block client from reading data until all nodes are successfully updated.

c) Data Locality: Reduce the physical distance between replicas to speed up data transfer.

4. Types of Consistency Models

Type	Definition	Behaviour	Use Case
Strong Consistency	The system does not allow reads until all node/replicas are updated	- Zero Dirty Reads. - Higher data integrity - Higher latency. (slower) because the system "locks" during updates	Payments, Banking, Ticket Booking
Eventual Consistency	The system allows read even if some nodes are not yet updated	- Possible Dirty - High availability and speed. - "Eventually" all nodes will become Consistent.	Social Media (Instagram/Facebook like or posts)
Weak consistency	There is no guarantee that all nodes will update.	- updates may or may not propagate to all nodes - Heavily dependent on specific business logic.	Real-time analytics, Voip/Video streaming

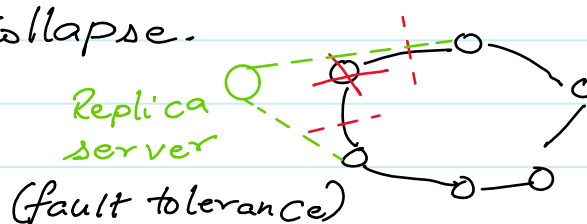
CAP Theorem

C - Consistency

A - Availability

P - Partition Tolerance

- For a distributed system, the CAP Theorem states that it is possible to **attain only two properties** and the third would be always compromised.
- The system requirement should define which two properties should be chosen over the rest.
- ☆ Consistency(C): All clients see the same data at the same time.
eg → If user A book a ticket user B must immediately see it as booked.
- ☆ Availability(A): Every requests gets a response (sucess/failure), even if data is slightly old.
eg → Google search or Youtube always loads, even if search index is 5 min old.
- ☆ Partition Tolerance: The system continues to work even if communication between servers (node) breaks.
eg → If a wire is cut or server crashes, the system must not collapse.



Crucial note: In any distributed system (like cloud/internet), network failures (partitions) are inevitable. Therefore, **Partition Tolerance (P)** is mandatory. You cannot choose CA in real distributed system.

Application	Classification	Reasoning
Blog Website	AP (Availability)	If a blog post doesn't load instantly for everyone, it's fine. It's more important that the site stays online.
Multiplayer Games (PUBG)	AP (Availability)	Speed/Uptime is critical. A slight lag in player position is acceptable compared to the game crashing.
Stock Trading	CP (Consistency)	Prices must be exact. Showing ₹5000 to one user and ₹10000 to another is disastrous. It's better to show "System Down" than wrong prices.
Video Streaming	AP (Availability)	If the site goes down, users leave. High availability is key to retention.
Ticket Booking (IRCTC)	CP (Consistency)	You cannot sell the same seat to two people (Double Booking). The system must lock seats strictly.
Video Chat	AP (Availability)	If WhatsApp goes down for 1 second, it's bad. It must always be up. Slight lag is okay.
Banking	CP (Consistency)	Accurate balance is non-negotiable. Banks often schedule downtime rather than risk inconsistent transactions.

Lambert Logical clock (Distributed systems)

Q. Why do we need it?

- The problem: In a distributed system (eg. server in USA, India, Australia), physical clocks are unreliable because

- △ Different time zones
- △ Network latency makes synchronization difficult
- △ Physical clocks drift (run faster/slower) over time.

- The Goal: We don't need to know the exact time (eg. 10 PM). We only need to **sequence of events** (i.e. Event A happened before B)

Lamport logical Clock

27 December 2025

09:52 PM

2. What is a Lamport logical clock?

⇒ It is a simple algorithm used to order events in a distributed system without using physical time.

⇒ Every process server maintains a simple counter initialized to 0 or 1. This counter acts as its clock.

Rules :

i> internal / Send Event :

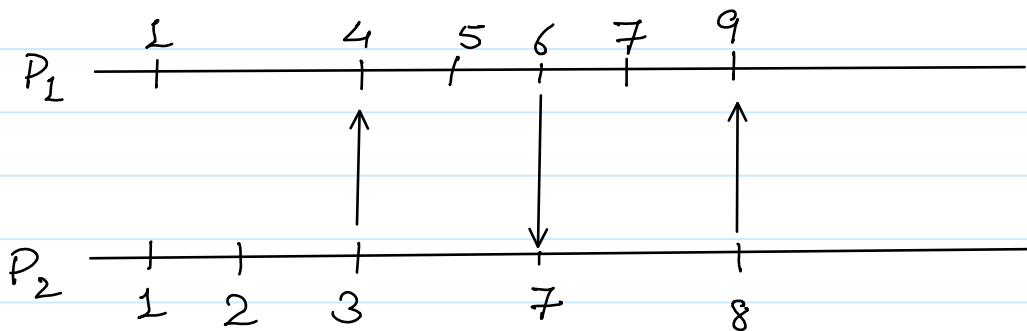
◦ Before any local event or sending a message, increment the counter

◦ Formula $\text{Counter} = \text{Counter} + 1$

ii> Receive Event :

◦ When receiving a message with a sender's timestamp (T_{sender}), update local counter to be higher than both

◦ Formula: $\text{Counter} = \max(\text{local_counter}, T_{\text{sender}}) + 1$



Vertical vs Horizontal Scaling

28 December 2025 01:55 PM

- ⇒ Vertical scaling: Increasing the power of a single machine or server.
- ⇒ How: Adding more RAM, upgrading the CPU, increasing hard disk space on the same server.

<u>Pros</u>	<u>Cons</u>
i> Easy Implementation	i> Single point of failure.
ii> Less Management: You have only one machine to manage.	ii> Hardware limit: There is limit how much ram/cpu we can put in a machine
iii> No Network Latency: Everything is in one box	iii> Cost: High-end "super-computer or mainframes are expensive.

- ⇒ Horizontal Scaling: Adding more machines to the pool to share the workload.

How: Instead of upgrading 1 server, you buy 4 new standard servers and distribute the traffic/data across them.

<u>Pros</u>	<u>Cons</u>
i> No single point of failure: If one crashes another take its place.	i> Complex management: Managing 100 server is harder than managing one.
ii> Unlimited growth: Theoretically, you can keep adding more servers	ii> Data Consistency: Keeping data across multiple server is difficult
iii> Cost effective: You can use cheaper hardware.	iii> Higher power/space

Redundancy and Replication

28 December 2025

02:15 PM

Redundancy: Redundancy is simply the duplication of nodes or components so that when a node or components fails, the duplicate node is available to service customers.

Types of Redundancy:

- i> Active Redundancy: Active Redundancy is considered when each unit is operating/active and responding to the action. Multiple nodes are connected to a load balancer, and each unit receives an equal load.
- ii> Passive Redundancy: Passive Redundancy is considered when one node is Active or operational and the other is not operating. During the breakdown of the active node, the passive node maintains availability by becoming the active node.

Replication: Redundancy + Synchronization

- ⇒ Context: Usually refers to Database (stateful servers)
- ⇒ Why sync: Unlike code servers, database change constantly (insert/update/delete).

Types of Replication:

- i> Active Replication (Multi-master)
 - All database nodes are active.
 - Any code can accept Read and Write requests.

Load Balancers

28 December 2025

02:32 PM

Challenge: Complex to Keep data synced in real-time.

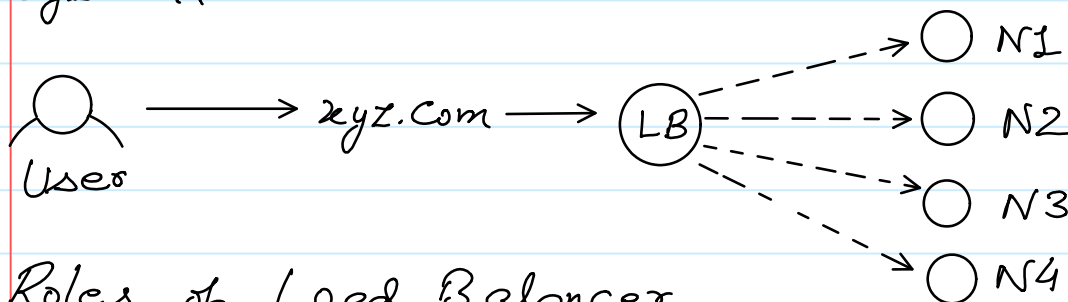
2 Passive Replication (Master-slave)

- Master Node: Handles Writes (Insert/update)
- Slave Node: Handle Reads (select). They get data updates from master.
- If master fail: One slave is promoted to become master.

⇒ Master-slave replication can either be synchronous or Asynchronous. The difference is simply the timing of propagation of changes. If changes are made to the master and slave at the same time, it is synchronous, If changes are queued up and written later, it is asynchronous

□ Load Balancers

⇒ Load Balancing is the process of efficient distribution of network traffic across all nodes in a distributed system.



◇ Roles of Load Balancer

1. The load distribution is equal over every node.
2. Health Check (if the node is not operational, the

Load Balancers

03 January 2026

02:03 PM

the request is passed to another node that is up and running.

3. Load Balancers ensure high scalability, high throughput and high availability.

☆ When to use it and when not to?

⇒ In monolithic or in case of vertically scaled system we don't need it. But in microservice architecture we need it.

□ Challenges of Load Balancing

a) Single point of failure - During a load balancer malfunctioning, the communication between clients and servers would be broken. To solve this issue we can use redundancy. The system can have an active load balancer and one passive load balancer.

****Advantages of using load balancers:****

1. ***Optimisation*:** Load Balancers help in resource utilisation and lower response time, thereby optimising the system in a high traffic environment.

2. ***Better User Experience*:** Load balancers help in reducing the latency and increasing availability, making the user's request go smoothly and error-free.

Advantages of using load balancer

12 January 2026

08:09 PM

3. ***Prevents Downtime:** Load Balancers maintain a record of servers that are non-operational and distribute the traffic accordingly, therefore ensuring security and preventing downtime, which also helps increase profits and productivity.

4. ***Flexibility:** Load Balancers have the flexibility to re-route the traffic in case of any breakdown and work on server maintenance to ensure efficiency. This helps in avoiding traffic bottlenecks.

5. ***Scalability:** when the traffic of a web application increases suddenly, load balancers can use physical or virtual servers to deliver the responses without any disruption.

6. ***Redundancy:** Load Balancing also provides in-build redundancy by re-routing the traffic in case of any failure.

****Load Balancing Algorithms****

1. Round Robin (Static) - rotation fashion.

2. Weighted Round robin (Static) - It is similar to the Round Robin when the servers are of different capacities. (some node can have better resources, others might not have)

3. IP Hash Algorithm (Static) - The servers have almost equal capacity, and the hash function (input is source IP) is used for random or unbiased distribution of requests to the nodes.

Load balancer

03 January 2026 02:44 PM

4. **Source IP Hash (Static)** - Source IP Hash combines the server and client's source and destination IP addresses to produce a hash key. The key can be used to determine the request distribution.
5. **Least Connection Algorithm (Dynamic)** - Client requests are distributed to the application server with the least number of active connections at the time the client request is received.
6. **Least Response Time (Dynamic)** - The request is distributed based on the server which has the least response time.

□ What is Caching ?

⇒ Caching is the process of storing copies of data or files in a temporary storage location (Cache) so they can be accessed faster.

Why use it? Fetching data from a primary database (Disk/SSD) is slow. Fetching data from a Cache (RAM/Memory) is extremely fast.

How It Works

1. **Request:** User asks for data (e.g., "Get Netflix Plans")
2. **Check Cache:** The system checks if the data is already in the Cache.
 - **Cache Hit:** Data found! Return it immediately. (Fastest)
 - **Cache Miss:** Data not found. Go to the Database, get the data, save a copy in the Cache, and then return it to the user.

Caching

03 January 2026

03:24 PM

3. **Invalidation:** If you change the data (e.g., update Netflix price), the system must delete the old cache so users don't see wrong info.

Types of Caching

a. In-Memory / Local Cache:

- Stored on the same server as the application.
- Fastest access but hard to share between multiple servers.
- Tools: Ehcache, Caffeine.

b. Distributed / External Cache:

- Stored on a separate server cluster shared by all application nodes.
- Best for distributed systems (Microservices).
- Tools: Redis, Memcached.

c. CDN (Content Delivery Network):

- Geographically distributed servers that cache Static Content (Images, CSS, Logos, Videos).
- Example: If a user in India opens an Amazon US site, the logo/images load from a server in Mumbai (CDN) instead of traveling from the USA.

Cache Eviction Strategies

03 January 2026 03:55 PM

1. Why do we need Cache Eviction?

- **Limited Size:** Cache memory (RAM) is expensive and limited. We cannot store everything forever.
- **Stale Data:** Data changes over time (e.g., Netflix plans change from 4 to 5). If we don't remove old data, users will see incorrect info.
- **Eviction:** The process of removing data from the cache to free up space for new data.

2. Cache Miss vs. Cache Hit

- **Cache Hit:** Data is found in the cache.
- **Cache Miss:** Data is NOT found in the cache. The system must fetch it from the Database, then save it to the cache for next time.

3. Eviction Policies (Algorithms)

When the cache is full and we need to add new data, which old data should we delete? These strategies decide that.

1. LRU (Least Recently Used):

- **Rule:** Remove the item that hasn't been used for the longest time.
- **Logic:** If you haven't used it recently, you probably won't use it soon.
- **Pros:** Most popular and generally efficient.
- **Cons:** Requires tracking "time since last use" (overhead).

2. LFU (Least Frequently Used):

- **Rule:** Remove the item that is used the least number of times (lowest count).

Cache Eviction Strategies

03 January 2026 04:01 PM

- Logic: If "Anime" is accessed 100k times and "Bollywood" only 50 times, delete "Bollywood" first.
- Pros: Keeps popular items safe.
- Cons: Needs to store a counter for every item.

3. MRU (Most Recently Used):

- Rule: Remove the item that was just added/used.
- Logic: Opposite of LRU. Used in specific scenarios (e.g., looping through a large dataset once where you won't need the recent item again soon).

4. FIFO (First In First Out):

- Rule: Remove the item that entered the cache first (Oldest created).
- Logic: Like a queue. The first one to come in is the first one to leave.
- Pros: Simple to implement.
- Cons: Might delete a popular item just because it's old.

5. LIFO (Last In First Out):

- Rule: Remove the item that entered the cache last (Newest created).
- Logic: Like a stack. The most recent item is deleted to make space.

6. RR (Random Replacement):

- Rule: Randomly select any item and delete it.
- Logic: No tracking needed.
- Pros: Very easy/fast to implement (no overhead).
- Cons: Unpredictable; might delete important data.

File Based Storage System (File Based DBMS)

03 January 2026

04:08 PM

1. Definition: A simple database management system where data is stored directly in the form of Files (like .txt, .csv) inside folders and sub-folders.

2. Disadvantages / Challenges

This system is rarely used for complex applications due to these major flaws:

a. Data Redundancy (Duplication):

- Issue : The same data might be stored in multiple places.
- Example : If two students (Ram & Shyam) are in the same college, the "College Details" file might be copied into both of their folders.
- Consequence: Waste of storage space.

b. Data Inconsistency:

- Issue : If data is duplicated, updating it in one place doesn't automatically update it elsewhere.
- Example : If the college address changes, you might update it in Ram's folder but forget Shyam's folder. Now you have two different addresses for the same college.
- Example : You delete a private photo from one folder but forget a copy exists in another folder (Delete Anomaly).

c. Poor Security:

- Issue: Harder to control access. If you have multiple copies of a file (due to redundancy), securing one copy doesn't guarantee the others are safe. A user might find the forgotten copy.

d. Slow Speed:

- Issue: Searching and retrieving data from flat files is much slower compared to structured databases (like SQL), especially as data grows.

RDBMS (Relational Database Management System)

03 January 2026 04:16 PM

Definition: A software that manages databases where data is stored in Tables (Rows & Columns) and these tables are related to each other (using Foreign Keys).

- Core Operations: Create, Read, Update, Delete (CRUD).
- Examples: MySQL, PostgreSQL, Oracle.

Advantages (Over File-Based Systems)

- No Redundancy: Using normalization and distinct tables (e.g., a "College" table) prevents duplicating data.
- Data Integrity: You can enforce rules (e.g., "This column must only contain numbers").
- Concurrency: Handles multiple users accessing data at the same time using Locking mechanisms (prevents conflicts).
- Built-in Searching: SQL queries allow powerful and fast data retrieval.

Disadvantages / Challenges

1. Rigid Schema:

- Problem: The structure is fixed. You define columns (ID, Name, City) upfront.
- Limitation: If you want to add a "Pan Card" field for just one user later, you must alter the entire table structure for everyone. It lacks flexibility compared to NoSQL.

2. High Cost: Managing complex RDBMS often requires expensive hardware and licensing.

3. Scalability Issues (The Biggest Problem):

- RDBMS is designed to scale Vertically (adding more RAM/CPU to one machine).
- It is very hard to scale Horizontally (Sharding/Distributing data across multiple machines).

NoSQL Databases

03 January 2026 09:54 PM

1. What is NoSQL?

- **Definition:** Stands for "Not Only SQL" or "Non-Relational Database."
- **Purpose:** Created to solve the limitations of RDBMS (Scaling issues, rigid schema).
- **Key Features:** Flexible schema (no fixed tables), easy horizontal scaling, and handles unstructured data.

2. Types of NoSQL Databases

There are 4 main categories under the NoSQL umbrella:

a. Key-Value Database :

- **Structure:** Stores data as a simple key and value pair. (Like a Hash Map).
- **Best For:** Simple data lookup, caching, Shopping Carts, Leaderboards.
- **Example:** Redis, DynamoDB.
- **Use Case:** Storing a user's shopping cart items or a game score.

b. Document Database :

- **Structure:** Stores data in JSON-like Documents.
- **Best For:** "Best of both worlds" – allows complex data structures (nested data) while remaining flexible. No rigid schema (you can add a new field to just one document).
- **Example:** MongoDB.
Use Case: E-commerce product catalogs (where different products have different attributes) or Content Management Systems.

c. Columnar Database

- **Structure:** Stores data by columns instead of rows.

NoSQL Databases

04 January 2026 01:14 AM

- **Benefit:** Extremely fast for analytical queries (e.g., "Find the average salary of all employees"). It only reads the specific column needed, unlike RDBMS which reads the whole row.
- **Best For:** Data Analytics, Big Data processing, Machine Learning.
- **Example:** Cassandra, HBase.
- **Use Case:** Analyzing logs or financial trends.

Graph Database

- **Structure:** Stores data as Nodes (Entities) and Edges (Relationships).
- **Benefit:** Designed for highly connected data. Finds relationships very quickly (e.g., "Friend of a Friend").
- **Best For:** Social Networks, Recommendation Engines, Maps.
- **Example:** Neo4j.
- **Use Case:** LinkedIn Connections ("2nd Degree Connection") or Google Maps routing.

3. Which Database to Choose? (Video Quiz)

Scenario	Recommended DB	Reasoning
LinkedIn / Social Network	Graph DB (Neo4j)	Data is highly connected (Friends, Connections).
Shopping Cart	Key-Value (Redis)	Simple data structure (Product ID + Qty) needs fast access.
Cricket Scorecard	Key-Value (Redis)	Simple mapping of Player Name -> Score.
Data Analysis / ML	Columnar (Cassandra)	Fast aggregation (Sum/Avg) on specific columns.
Payments / Banking	RDBMS (MySQL)	Crucial: Financial data needs ACID compliance and strong consistency, which RDBMS provides better than NoSQL.

Polyglot Persistence

04 January 2026

01:30 AM

1. What is Polyglot Persistence?

- **Definition:** The practice of using multiple types of databases within a single application to handle different kinds of needs effectively.
- **Concept:** Instead of forcing one database (like MySQL) to do everything, you pick the "best tool for the job" for each specific feature.
- **Analogy:** Just as a "polyglot" person speaks multiple languages, a "polyglot persistence" architecture speaks to multiple databases.

2. Why use it?

- A single database type (e.g., RDBMS) might not be efficient for all requirements.
- Some data needs high speed (Caching), some needs flexible schema (Orders), some needs strict consistency (Payments), and some needs relationships (Social Graph).
- By mixing them, you optimize performance and scalability.

3. Example: E-Commerce Architecture

An E-commerce app (like Amazon/Flipkart) using Polyglot Persistence might look like this:

Feature	Recommended Database	Database Type	Reasoning
Shopping Cart	Redis	Key-Value	Needs super-fast access for temporary data. Simple structure (Item ID + Qty) .
Order History	MongoDB	Document	Orders have complex, nested details (Address, Items, Status) that vary. Flexible schema is best .
Inventory & Payments	PostgreSQL / MySQL	RDBMS	Financial data requires strict ACID transactions and consistency. You can't risk money errors .
User Social Graph	Neo4j	Graph	If users follow each other or you want "People who bought this also bought that" recommendations .
Analytics	Cassandra	Columnar	For analyzing massive amounts of sales logs

Denormalization

04 January 2026 01:41 AM

1. What is Normalization?

- **Concept:** Dividing a single large table into multiple smaller tables to reduce **data redundancy** (duplication).
- **Example:** Instead of repeating "Dept Name" & "Description" for every employee (100 rows), we create a separate Department table and just use Dept_ID in the Employee table.
- **Goal:** Save space and ensure data consistency.

2. What is Denormalization?

- **Definition:** The process of combining multiple tables back into a **single table** by adding redundant data.
- **Concept:** It is the *opposite* of normalization. You intentionally introduce duplication.
- **Goal:** To optimize **Read Performance (Speed)** by removing the need for complex JOINS.

3. Why use Denormalization? (Benefits)

a. Faster Reads:

- To get "Employee Name" and "Dept Description" in a normalized DB, you must **JOIN** two tables (Employee + Department). This is slow.
- In a denormalized DB, "Dept Description" is already in the Employee row. You read it instantly.

b. Less Complexity: No need to write complex SQL queries with multiple Joins.

c. Fewer Network Calls: You fetch all needed data in one go from one table, rather than querying multiple tables or checking multiple servers.

Database Indexing

04 January 2026

01:47 AM

4. Challenges / Disadvantages

- a. **Data Redundancy:** Wastes storage space because "Dept Description" is repeated for every employee.
- b. **Data Inconsistency:** If you update the "Dept Name" in one row but forget to update it in the other 99 rows for the same department, your data becomes corrupt (Inconsistent).
- c. **Slow Write Operations:** Updating data is slower because you have to update it in **multiple places** (every row where that department exists) instead of just one place (the Department table).

Database Indexing

1. The Problem: Unorganized Data:

- **Analogy:** Finding a key in a messy room vs. finding medicine in an organized pharmacy.
 - **Unorganized (No Index):** You have to search everything one by one (Linear Search, $O(N)$).
 - **Organized (With Index):** You know exactly where to look (Binary Search, $O(\log N)$).

2. How Indexing Works:

- **Concept:** Indexing creates a separate data structure (like a Lookup Table) that stores the value of a specific column in a **Sorted Order** along with a pointer to the original row.
- **Structure:** [Sorted Value | Pointer to Row]
- **Underlying Data Structure:** Most databases use **B-Trees** or **B+ Trees** for indexing.

3. Example Scenario

- **Table:** Students (1 Million Rows).
- **Query:** `SELECT * FROM Students WHERE NetWorth = 4000.`

Database Indexing

04 January 2026 02:05 AM

- **Without Index:**
 - The database scans every single row from 1 to 1,000,000 to find who has 4000.
 - Time Complexity: $O(N)$ (Slow).
- **With Index on NetWorth:**
 - The DB looks at the *Index Table* (which is sorted: 300, 1000, 4000, 10000...).
 - It uses Binary Search to jump straight to "4000".
 - It takes the pointer and fetches the full row immediately.
 - Time Complexity: $O(\log N)$ (Fast).

4. When to Use Indexing? (Best Practices)

- **YES:** Use on *Read-Intensive* tables (tables where you search/select often).
 - Use on columns frequently used in WHERE, JOIN, or ORDER BY clauses.
 - Use on large datasets (Millions of rows).
- **NO:** Do NOT use on *Write-Intensive* tables (tables with frequent INSERT, UPDATE, DELETE).
 - **Reason:** Every time you add a new row, the database has to update the main table **AND** the Index table (and re-sort it). This slows down writing speed significantly.
 - Do not use on small tables (scanning 100 rows is fast enough without an index).

Synchronous Communication

05 January 2026 09:21 PM

Definition: A communication model where the sender sends a request and **waits** (gets blocked) until it receives a response from the receiver.

Key Characteristic: It is a **Blocking Call**. You cannot perform other tasks while waiting.

Sequential Process: Steps happen one after another. Step B cannot start until Step A is finished.

Real-World Analogies

- **ATM Machine:**

- You insert card then Enter PIN then **Wait** for processing then Collect Cash.
- You cannot leave or do anything else until the cash comes out. You are "blocked".

When to Use It? (Use Cases)

- **To Achieve Consistency:**

- If you update data in a Primary Database, you want to ensure it is updated in all Replica Databases before confirming success to the user. This ensures everyone sees the same data (High Consistency).

- **Transactions (Payments):**

- When transferring money, step 1 (Debit) and step 2 (Credit) must happen in order. You need immediate confirmation.

Industrial Examples

1. **Stock Market:** Prices must be consistent for everyone instantly.
2. **Banking/Payments:** You need to know immediately if a payment failed or succeeded.

Asynchronous Communication

05 January 2026 09:32 PM

Definition: A communication model where the sender sends a request but does not wait for the response. It can continue doing other work while the request is being processed.

Key Characteristic: It is a Non-Blocking call.

Contrast: Unlike synchronous (sequential steps), asynchronous steps do not block the main flow.

Real-World Analogy (Mukhiya Ji & Bakery)

- **Scenario:** Mukhiya Ji goes to a bakery to buy a cake.
- **Baker:** "It will take 3 hours to bake."
- **Synchronous Approach:** Mukhiya Ji stands there for 3 hours doing nothing, just waiting for the cake. (Inefficient).
- **Asynchronous Approach:** Mukhiya Ji says, "Okay, I'll come back later." In those 3 hours, he goes to do farming, read, or milk the buffalo.
- **Outcome:** He utilized the waiting time for other productive tasks instead of being blocked.

E-Commerce Example (Amazon)

Feature	Type	Reason
Add to Cart	Synchronous	You need to know immediately if the item is in stock. You can't say "Added" and then fail later.
Payment	Synchronous	Money transfer requires high consistency. You wait for success/failure confirmation.
Order Email	Asynchronous	The "Order Placed" email isn't urgent. If it comes 2 minutes later, it doesn't stop the user from browsing